

Sinothando MAFU

05 October 2026

SQL Exercise 2

Aggregate Functions & Operators

Section 1

Q1:

```
SELECT DISTINCT department  
FROM students;
```

R: students

department
IT
HR
Finance

Q2:

```
SELECT department,  
AVG(age) AS avg_age  
FROM students  
GROUP BY department;
```

R: students

department	avg_age
IT	20.5
HR	22
Finance	23

Q3:

```
SELECT department,  
COUNT(name) AS student_count  
FROM students  
GROUP BY department
```

HAVING COUNT (name) > 1;

R: students

department	student count
IT	2
HR	2

Q4:

```
SELECT student_id,  
       name ,  
       age ,  
       department  
FROM students  
WHERE Age BETWEEN 21 AND 23;
```

R: students

student_id	name	age	department
2	Bob	22	HR
3	Charlie	21	IT
4	Diana	23	Finance
5	Eve	22	HR

Q5:

```
SELECT student_id,  
       name ,  
       age ,  
       department  
FROM students  
WHERE department IN ('IT', 'HR')  
AND age > 21;
```

R: students

student_id	name	age	department
2	Bob	22	HR
5	Eve	22	HR

Section 2

Q6:

```
SELECT department,  
       SUM(credits) AS total_credits  
FROM courses  
GROUP BY department  
HAVING SUM(credits) > 5;
```

R: courses

department	total_credits
IT	11

Q7:

```
SELECT *  
FROM courses  
WHERE credits != 4;
```

R: courses

course_id	course_name	department	credits
101	SQL Basics	IT	3
104	Excel	Finance	2
105	Statistics	HR	3

Q8:

```
SELECT course_id,  
       course_name,  
       credits  
FROM courses  
ORDER BY credits DESC  
LIMIT 3;
```

R: courses

course_id	course_name	credits
102	Python	4
103	Data Science	4
101	SQL Basics	3

Section 3

Q9:

```
SELECT MAX(grade) AS max_grade,  
       MIN(grade) AS min_grade,  
       AVG(grade) AS Avg_grade  
FROM enrollments;
```

R: enrollments

max_grade	min_grade	avg_grade
90	78	84.6

Q10:

```
SELECT course_id,  
       COUNT(enrollment_id) AS enrollment_count  
FROM enrollments  
GROUP BY course_id;
```

R: enrollments

course_id	enrollment_count
101	1
102	1
103	1
104	1
105	1

Section 4:

Q11:

```
SELECT department,  
       SUM(salary) AS total_salary,  
       SUM(bonus) AS total_bonus
```

```
FROM salaries
```

```
GROUP BY department;
```

R: salaries

department	total_salary	total_bonus
IT	122000	10500
HR	109000	7500
Finance	70000	6000

Q12:

```
SELECT department,  
       AVG(salary) AS avg_salary
```

```
FROM salaries
```

```
GROUP BY department
```

```
HAVING AVG(salary) > 55000;
```

R: salaries

department	avg_salary
------------	------------

R: salaries

department	avg_salary
IT	61000
Finance	70000

Q13:

```
SELECT employee_id,  
       name,  
       salary,  
       bonus,  
       salary + bonus AS total_compensation  
FROM salaries  
WHERE salary + bonus > 60000;
```

R: salaries

employee_id	name	salary	bonus	total_compensation
1	Tom	60000	5000	65000
3	Spike	70000	6000	76000
4	Tyke	62000	5500	67500

Section 5

Q14:

```
SELECT department,  
       SUM(budget) AS total_budget,  
       AVG(budget) AS avg_budget  
FROM projects  
GROUP BY department  
HAVING AVG(budget) > 70000;
```

R: projects

department	total_budget	avg_budget
IT	270000	135000
Finance	80000	80000

Q15:

SELECT *

FROM projects

WHERE budget BETWEEN 50000 AND 120000

AND NOT department = 'Marketing';

R: projects

project_id	project name	department	budget
1	AI App	IT	120000
2	Payroll System	Finance	80000
5	HR Portal	HR	50000